DATA SCIENCE & ANALYTICS · SESSION 12

Classification & Ensembles

Predicting a label — from logistic regression to Random Forest, XGBoost, and SHAP

Asst.Prof.Dr. Chaiwat N. · 2026

2026 · Bachelor Program · 3 hrs · Workshop Format · Hands-on Coding

Session Agenda

01 **The Classification Problem**
Predicting customer churn

02 **Logistic Regression**
Probabilities, thresholds, imbalance

03 **Trees & Ensembles**
From one tree to Random Forest and XGBoost

04 **The Right Metrics**
Why accuracy lies; precision, recall, ROC-AUC

05 **Explaining with SHAP**
Why the model made this prediction

06 **Workshop Block**
Build, evaluate, and explain a churn model

01

PART 01

The Classification Problem

Predict which customers will churn in the next 30 days — a yes/no label, not a number. The output is a probability you turn into a decision.

Classification predicts a category — and a probability behind it

The task

The goal	Predict a label: will this customer churn, yes or no? The model outputs a probability you threshold into a decision.
Why it's different	Regression predicts how much; classification predicts which. Different models, and crucially different metrics.
Class imbalance	Churners are rare — maybe 10%. A model that predicts 'nobody churns' is 90% accurate and totally useless.
Builds on S10–11	Same pipeline and split; new models and a new metric toolkit built for labels.

Source: DS Course, Session 12, 2026

02

PART 02

Logistic Regression

Despite the name, logistic regression is the baseline classifier: it outputs a probability, and you choose the threshold that turns it into a yes or no.

Logistic regression: a probability, then a threshold

The classification baseline

Probability out

predict_proba gives each customer a churn probability 0–1, not just a hard label — far more useful.

Threshold

Above 0.5 means 'churn' by default, but you set it. Lower it to catch more churners at the cost of false alarms.

Handle imbalance

class_weight='balanced' tells the model rare churners matter as much as the common stayers.

PYTHON · LOGISTIC

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(
    class_weight='balanced', max_iter=1000)
clf.fit(X_train, y_train)

proba = clf.predict_proba(X_test)[:, 1]
preds = (proba > 0.3).astype(int)    # custom cut
```

predict_proba + your threshold = the decision.

From one tree to an ensemble: Random Forest and XGBoost

Many weak learners, one strong model

One tree is fragile

A single decision tree overfits and shifts wildly with small data changes. Powerful but unstable alone.

Random Forest

Averages many trees on random subsets — bagging. Far more stable and accurate than any single tree.

XGBoost

Builds trees in sequence, each fixing the last's mistakes — boosting. Often the top performer on tabular data.

PYTHON · ENSEMBLES

```
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

rf = RandomForestClassifier(
    n_estimators=300, class_weight='balanced')
rf.fit(X_train, y_train)

xgb = XGBClassifier(n_estimators=300)
xgb.fit(X_train, y_train)
```

Forests average; boosting corrects. Both beat one tree.

03

PART 03

The Right Metrics

On imbalanced problems accuracy is a trap. Precision, recall, the confusion matrix, and ROC-AUC tell you what the model is really doing.

Accuracy lies on rare events — use the metrics built for them

Measure what matters

Confusion matrix	The four outcomes: true/false positives and negatives. Every other metric is computed from this grid.
Precision vs recall	Precision: of those flagged churn, how many really were. Recall: of all real churners, how many we caught.
The trade-off	Lowering the threshold raises recall but lowers precision. Which costs more — a missed churner or a false alarm?
ROC-AUC	One number for ranking quality across all thresholds. 0.5 is coin-flip, 1.0 is perfect. Robust to imbalance.

Source: DS Course, Session 12, 2026

04

PART 04

The Right Metrics

See the metrics in code — the confusion matrix, the full report, and the AUC that compares models fairly.

Computing the metrics that actually matter

The evaluation toolkit

classification_report

Precision, recall, and F1 per class in one call. Read the churn row, not the overall accuracy.

ROC-AUC

roc_auc_score on the predicted probabilities ranks models fairly even when classes are imbalanced.

Compare models

Score logistic, forest, and XGBoost on the same test set and AUC. Let the metric pick the winner.

PYTHON · METRICS

```
from sklearn.metrics import (
    classification_report, roc_auc_score,
    confusion_matrix)

print(confusion_matrix(y_test, preds))
print(classification_report(y_test, preds))
roc_auc_score(y_test, proba)
```

Read the churn row — never the headline accuracy.

05

PART 05

Explaining with SHAP

An accurate model nobody can explain won't be trusted or deployed. SHAP shows exactly how much each feature pushed a given prediction up or down.

SHAP: explain why the model made this prediction

Open the black box

The question

'Why does the model think THIS customer will churn?' SHAP attributes the prediction to each feature, in plain units.

Summary plot

Ranks features by overall impact and shows direction — which drivers raise churn risk across all customers.

Waterfall

For one customer, shows each feature's push up or down from the baseline to the final prediction. Explainable AI.

PYTHON · SHAP

```
import shap

explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test)

# global: which features matter most
shap.summary_plot(shap_values, X_test)

# local: explain one prediction
shap.plots.waterfall(shap_values[0])
```

SHAP turns 'the model said so' into a reason you can give.

06

Workshop Block

Build a churn classifier, evaluate it with the right metrics, and explain one prediction with SHAP — the full modern classification workflow.

PART 06

By the end of this block, every student has a churn model that's evaluated and explained

4 Steps · Workshop

Build	Fit logistic regression and a Random Forest in your pipeline, handling the class imbalance.
Evaluate	Confusion matrix, classification_report, ROC-AUC. Compare the models on the churn class, not accuracy.
Tune the threshold	Pick a cutoff that fits the business cost of a missed churner vs a false alarm. Justify it.
Explain	Run a SHAP summary plot and one waterfall. State, in a sentence, why one customer was flagged. Your deliverable.

Source: Workshop, Session 12, 2026

KEY TAKEAWAY

A model that predicts well and explains why is one you can actually deploy.